NVR PIPELINE IMPLEMENTATION GUIDE

A simple, developer-readable view of the complete NVR data flow, user flow, implementation path, resource validation and references

Baseline workload	8 IP cameras • H.264/H.265 • 1080p class
Memory target	4 GB RAM class
AI capacity	Dynamic 2–3 concurrent AI execution capacity
CPU target	Sustained CPU utilization target < 50%
Main design rule	Recording stays compressed; AI/live use selected decoded frames
Version	2.0 • 06 October 2026

Important: The architecture is the selected implementation structure. Final workload claims must be proved by measurements on the actual target platform. Hardware-specific decoder/NPU behavior, queue sizes and performance values remain qualification parameters.

Contents

1. What We Are Building
2. Complete NVR Pipeline
3. How the Pipeline Works — Simple Explanation
4. User-Perspective Flow: What the User Does and What the NVR Does
5. Pipeline Components — One-Line Definitions
6. Camera Input Pipeline
7. Recording Pipeline
8. Live View Pipeline
9. Fullscreen Pipeline
10. AI Pipeline
11. Event and Rule Pipeline
12. Playback, Search and Export
13. Storage and Metadata
14. Voice Control Pipeline
15. UI, API and Unified Command Flow
16. Resource and Health Management
17. Data Flow Between Pipelines
18. How Developers Should Implement It
19. Important Developer Rules
20. Resource Validation and Medium-Detail Mathematics
21. Math Terms — Short Definitions
22. Validation Checklist
23. References and Resources
24. What Comes From Each Reference

1. What We Are Building

The NVR continuously receives video from IP cameras, records the compressed camera streams, shows live video, supports fullscreen and playback, runs AI on selected decoded frames, converts detections into events, stores searchable metadata, accepts commands from the UI/API/voice service, and manages storage and system resources.

The simplest way to understand the system is: camera video comes in → the NVR decides where that video is needed → recording keeps it compressed → live view and AI decode only what they need → results become events and searchable metadata → the user controls the system through one command path.

2. Complete NVR Pipeline

The diagram below is the same complete full-pipeline diagram used in the implementation guide. It has only been rotated 90° so the complete flow runs horizontally. No nodes, labels, connections or content were changed.

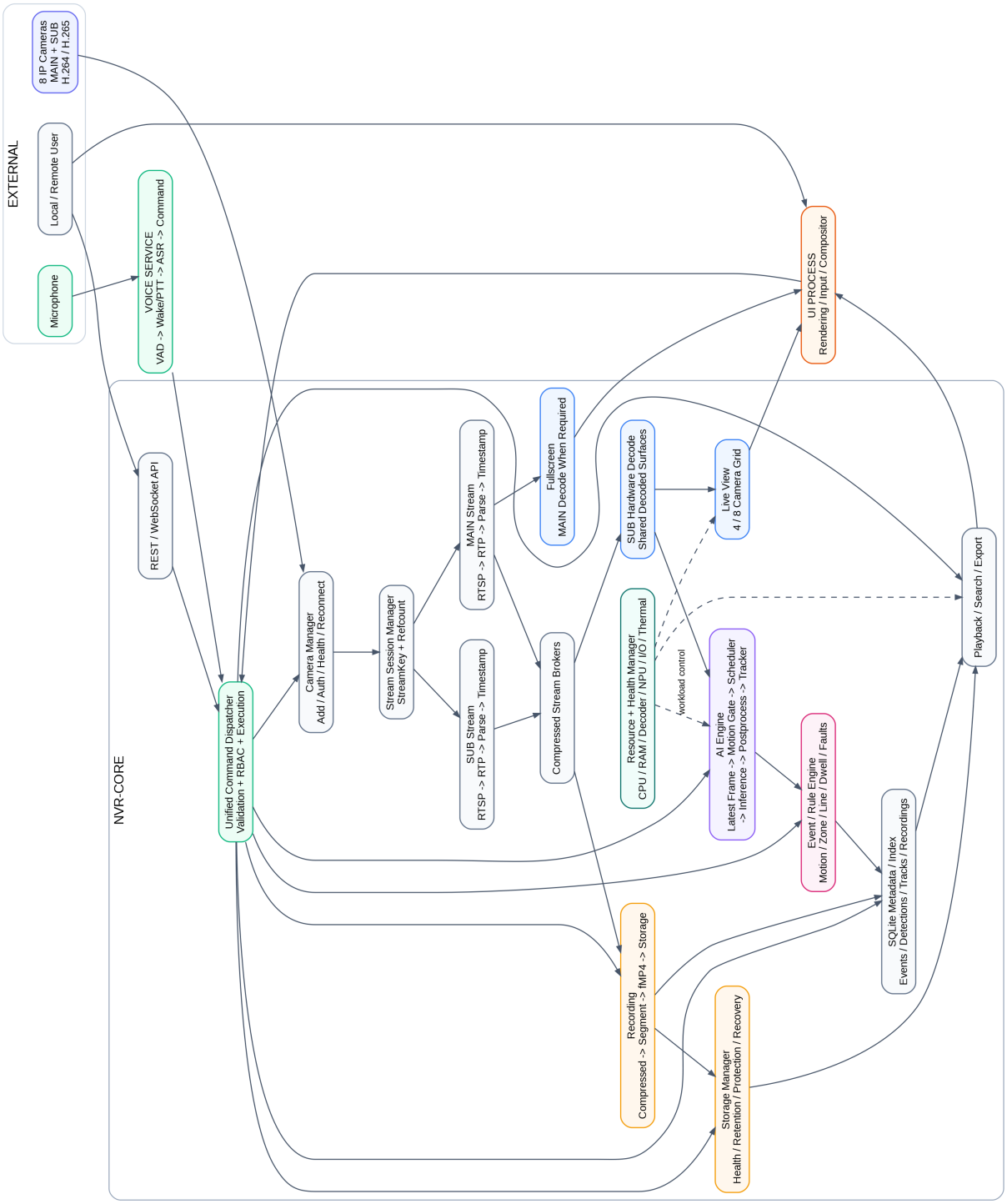


Figure 1. Complete NVR pipeline and control flow — the original diagram rotated 90° only; no nodes, labels or connections changed.

3. How the Pipeline Works — Simple Explanation

Think of the NVR as a set of connected paths. The camera sends a compressed video stream. The NVR first receives and organizes that stream. From there, different consumers use the same camera data for different jobs.

Path	What happens
Camera → Stream session	The NVR connects to the camera, receives RTSP/RTP data and keeps timing/codec information.
Stream → Recording	The compressed MAIN H.264/H.265 stream is segmented and stored. It normally does not need decoding.
SUB → Live	The SUB stream is decoded for the 4/8-camera live grid. Old live frames can be dropped to keep the display
SUB → AI	Selected decoded frames enter a latest-frame mailbox, motion gate, AI scheduler, inference, tracking and ev
Recording → Playback	When the user searches, metadata identifies the required recording segment; playback opens and decodes t
AI → Event → Metadata	Detections become tracked states and rule results. The NVR stores event/detection/track metadata and refer
UI/API/Voice → Command	All user commands are validated and authorized by the same command dispatcher before execution.
Resource manager → Pipelines	Measured CPU, RAM, decoder, AI, network, storage and thermal state influence background work and AI worl

The important idea

One camera does not mean one giant pipeline. The NVR keeps acquisition, recording, live display, AI, playback and control logically separate so one workload does not unnecessarily block another.

4. User-Perspective Flow: What the User Does and What the NVR Does

This section follows the system from the user's point of view. It answers: "I click something in the UI — what happens inside the NVR?"

A. User opens Live View

UI opens the live page → UI asks NVR core for the selected layout → core checks required camera streams → SUB sessions are acquired/reused → SUB frames are decoded → live frames reach the display/compositor → user sees the 4/8-camera grid.

B. User opens one camera fullscreen

User selects camera 4 → UI sends SHOW_CAMERA(4) → command dispatcher validates it → core checks whether MAIN acquisition already exists → if needed it creates one MAIN session → MAIN is decoded for fullscreen → UI displays the camera.

C. User starts recording manually

User presses Record → UI sends START_RECORDING(camera) → dispatcher validates permission → recorder attaches to the MAIN compressed stream → segment is written → finalized → indexed in SQLite → UI shows recording state.

D. User enables AI on camera 4

User selects a model → UI sends ENABLE_AI → dispatcher validates model/camera/permission → AI scheduler checks available capacity → decoded SUB frames enter the AI mailbox → motion gate/scheduler/inference/tracking run → detections are stored and rules can create events.

E. User searches an event

User selects camera/date/event filters → UI sends SEARCH_EVENTS → SQLite searches metadata → matching event and recording references are returned → user selects a result → playback opens the referenced segment → decoder seeks to the required time.

F. User uses voice

User speaks → voice service performs VAD/wake/PTT and ASR → parser converts speech to structured intent → command is validated and authorized → same command dispatcher executes it → UI/core state changes normally.

G. Camera goes offline

Camera connection fails → stream session reports failure → camera manager marks it offline → reconnect attempts run for that camera → recording/live/AI consumers for that camera receive state changes → other cameras continue.

H. System becomes resource constrained

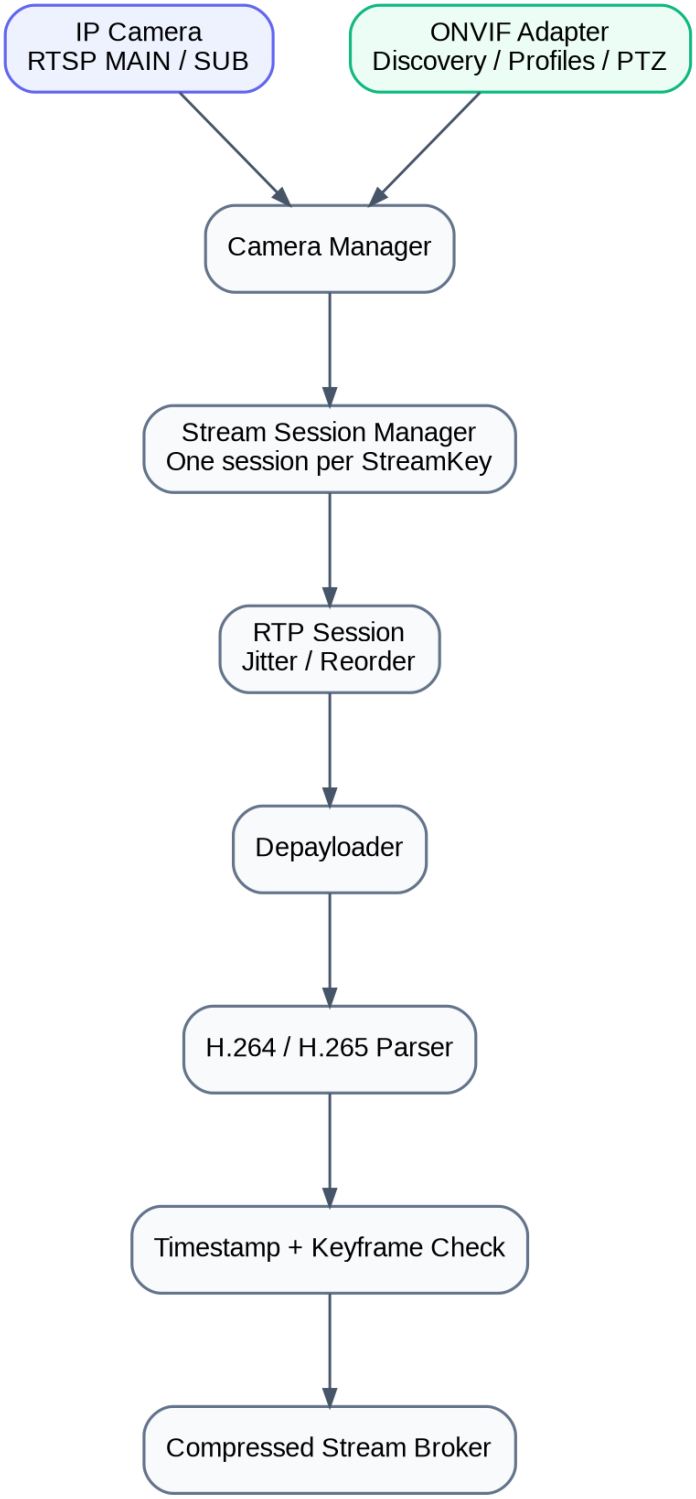
Resource manager measures pressure → controller reduces lower-priority work such as AI FPS/background playback → stale AI work is dropped → protected recording/ingest is preserved as long as the defined policy allows.

5. Pipeline Components — One-Line Definitions

Component	Simple meaning
Camera Manager	Stores camera settings and controls camera connection state, discovery, profiles and PTZ.
Stream Session Manager	Creates and reuses actual camera media sessions so the same stream is not opened unnecessarily.
Compressed Stream Broker	Makes the received compressed stream available to the required consumers.
Recording	Turns compressed MAIN video into finalized recording segments and indexes them.
Hardware Decoder	Converts compressed H.264/H.265 video into frames when live view, AI or playback needs decoded v
Live View	Builds the current 4/8-camera display from decoded SUB frames.
Fullscreen	Shows one selected camera, using MAIN when required and available.
AI Engine	Selects fresh frames, runs inference, tracks objects and produces analytics results.
AI Scheduler	Decides which AI jobs can run within measured inference capacity.
Event/Rule Engine	Turns detections and system states into meaningful events such as zone entry or camera failure.
Playback	Reads stored segments and decodes them for seeking, viewing and export.
Storage Manager	Monitors disk state, retention, protection, allocation and recovery.
SQLite Metadata	Stores indexes, events, detections, tracks, users, health and recording references.
Command Dispatcher	Receives UI/API/voice commands, validates them, checks permissions and executes them.
Resource Manager	Measures system load and controls workload when resources become constrained.
Voice Service	Converts speech into validated structured commands without directly controlling media pipelines.
UI Process	Displays video and controls and sends commands; it does not own RTSP acquisition.

6. Camera Input Pipeline

The camera pipeline establishes the connection and converts camera network traffic into usable compressed media. It also keeps codec, timing and health information so the rest of the NVR knows what it is receiving.



Pipeline diagram: 6. Camera Input Pipeline

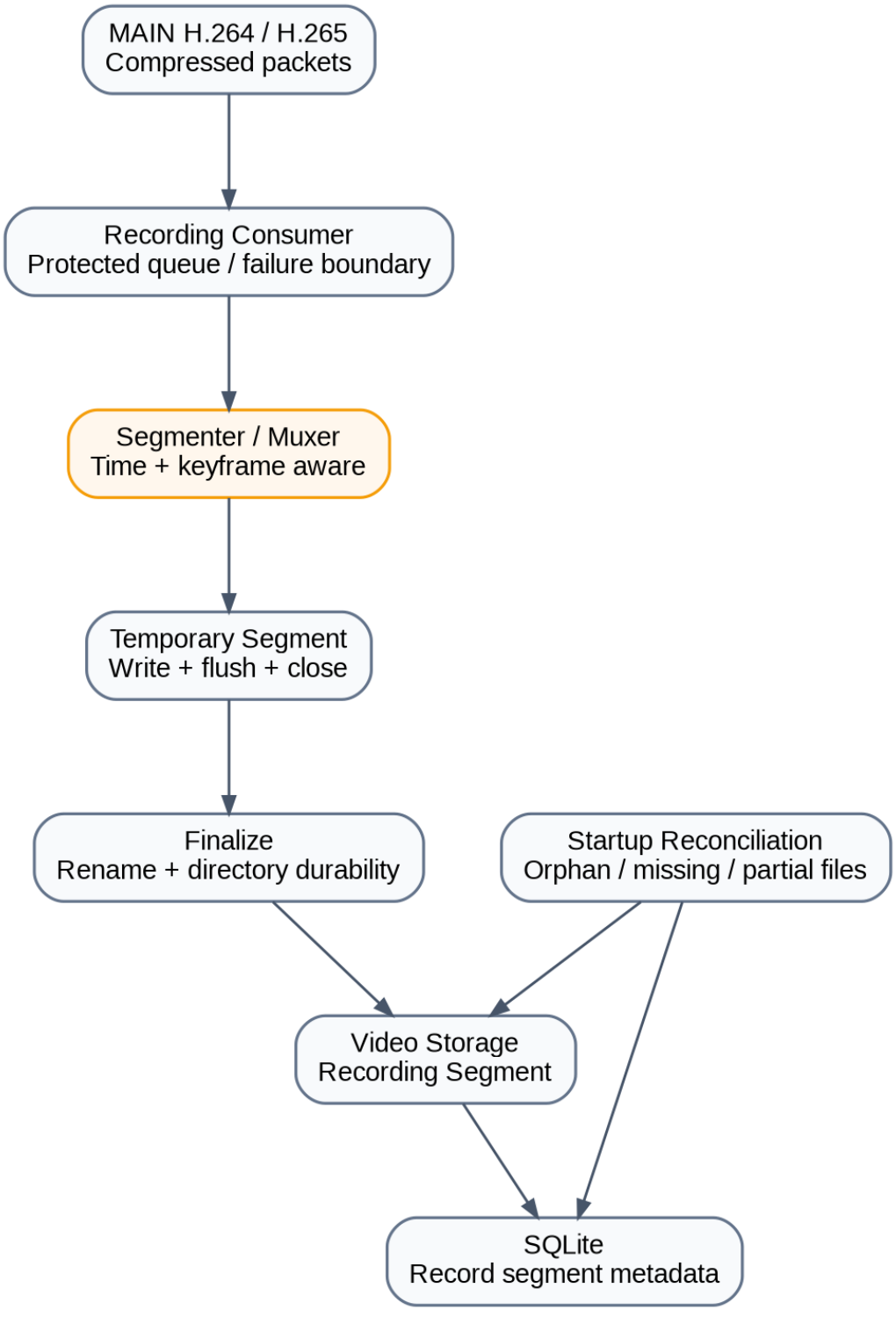
How it works

- Add camera and credentials.
- Discover with ONVIF when supported, or configure RTSP manually.
- Select MAIN and SUB stream profiles.
- Create the stream session only when a consumer needs it.
- Receive RTSP/RTP and handle jitter/reordering/timestamps.

- Expose stream and connection health.

7. Recording Pipeline

Recording is deliberately simple: use the compressed MAIN stream, split it into segments, finalize each segment safely, and store an index entry. Normal recording does not decode and re-encode the video.



Pipeline diagram: 7. Recording Pipeline

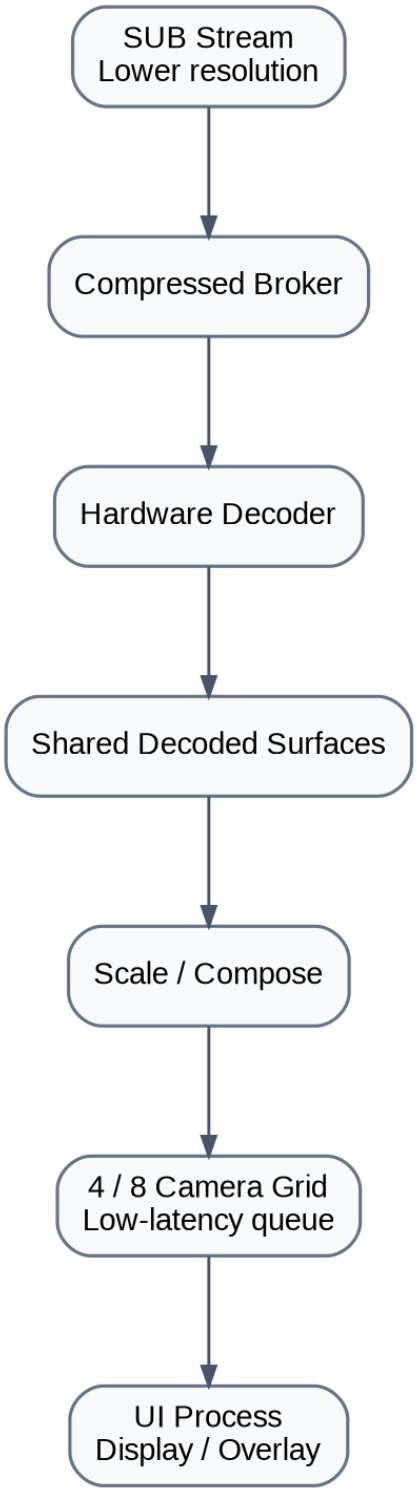
How it works

- Receive compressed MAIN packets.
- Check codec configuration, timestamps and keyframe/GOP boundaries.

- Write a temporary segment.
- Flush and close it.
- Finalize the file and move it to the final location.
- Commit the SQLite segment record.
- On startup, reconcile files and metadata.

8. Live View Pipeline

The live grid is designed for freshness. SUB streams are decoded and displayed, and bounded queues prevent old frames from building up latency.



Pipeline diagram: 8. Live View Pipeline

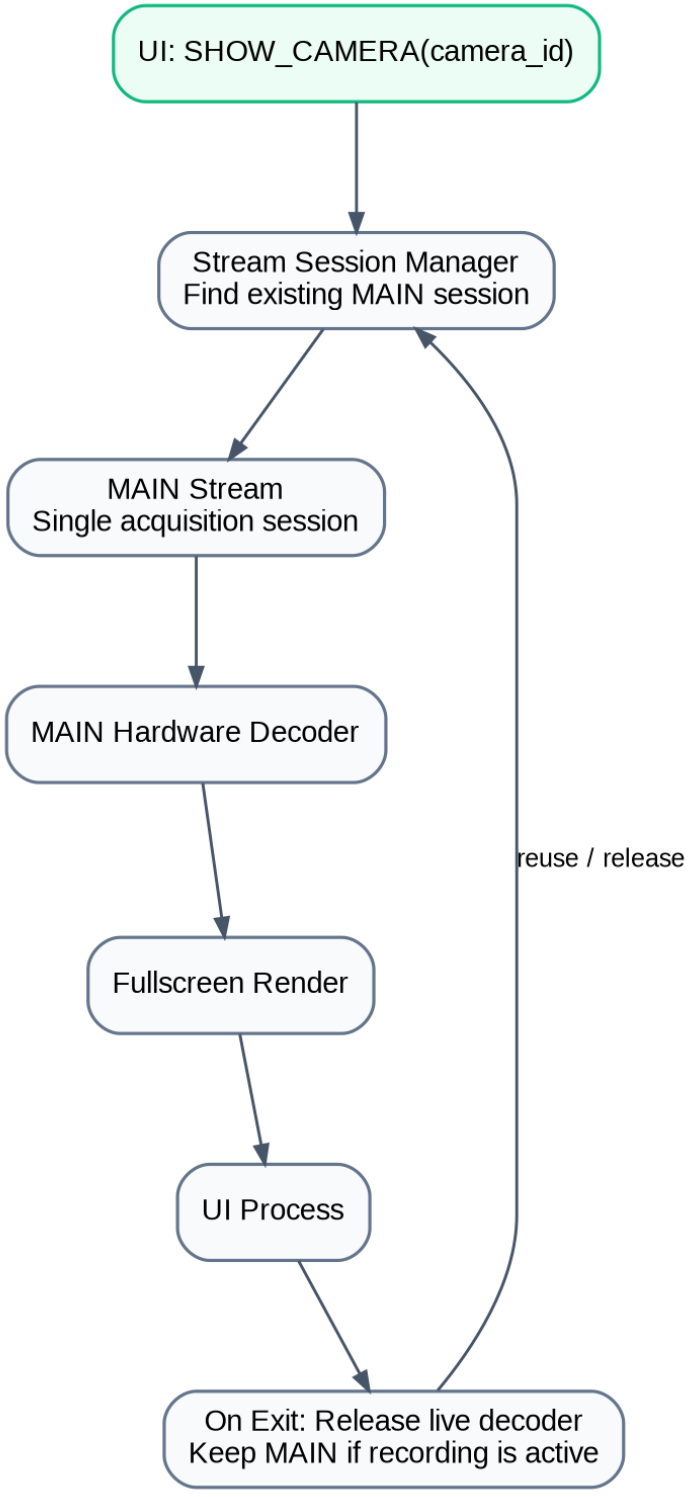
How it works

- UI requests a layout.
- Core identifies the required cameras.

- SUB sessions are acquired/reused.
- Hardware decoder produces frames.
- Frames go through the display path/compositor.
- Old frames may be dropped when necessary to keep the display current.

9. Fullscreen Pipeline

Fullscreen is a view change. It should not create unnecessary duplicate camera acquisition. The UI asks the core to show one camera, and the core reuses an existing MAIN session when possible.



Pipeline diagram: 9. Fullscreen Pipeline

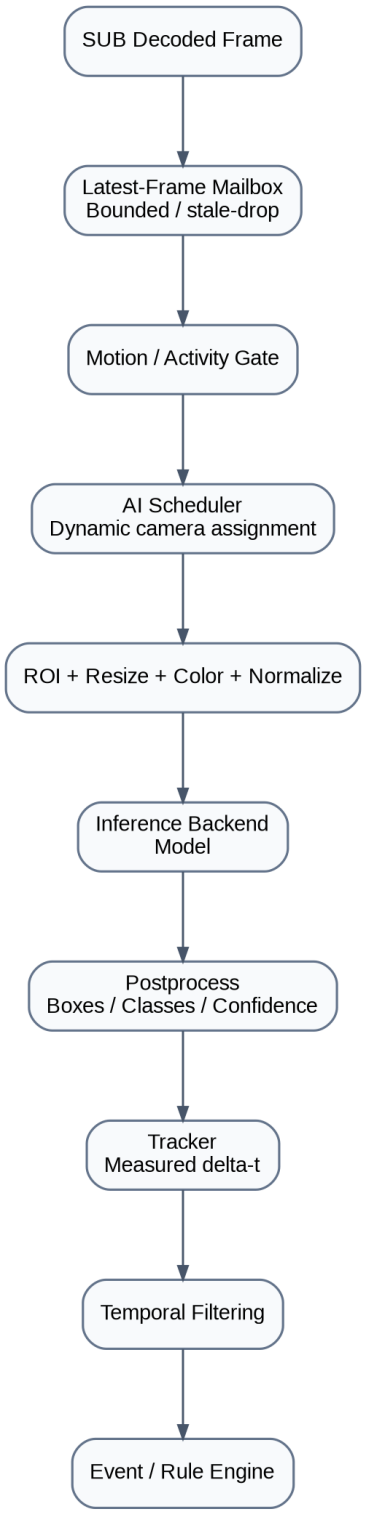
How it works

- UI sends SHOW_CAMERA(camera_id).
- Dispatcher validates the request.

- Core checks for an existing MAIN session.
- Reuse it if present; otherwise create one MAIN session.
- Decode the required video.
- Send the fullscreen result to the UI/display path.

10. AI Pipeline

AI uses decoded SUB frames and prioritizes fresh work. Any camera can be assigned AI. The scheduler decides when a frame can run based on measured capacity.



Pipeline diagram: 10. AI Pipeline

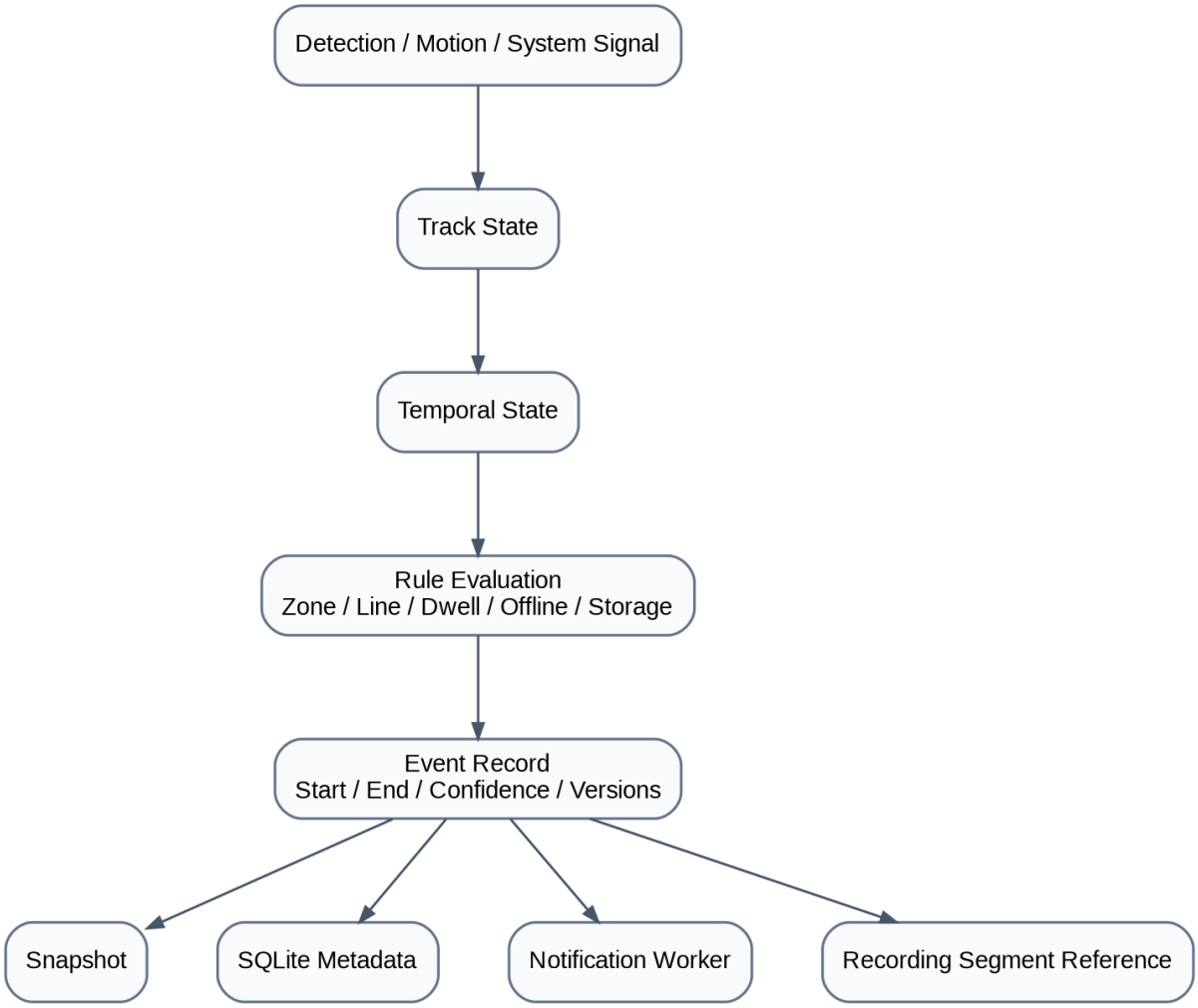
How it works

- Receive decoded SUB frame.
- Replace stale work in a bounded latest-frame mailbox.

- Apply optional motion/activity gating.
- Acquire an AI execution slot.
- Apply ROI/crop, resize, color conversion and normalization.
- Run the model.
- Postprocess detections.
- Track objects.
- Apply temporal filtering.
- Evaluate rules and store results.

11. Event and Rule Pipeline

An event is not just one detection. The event engine combines detections, tracking and time-based rules to create a meaningful state such as “person entered zone”.



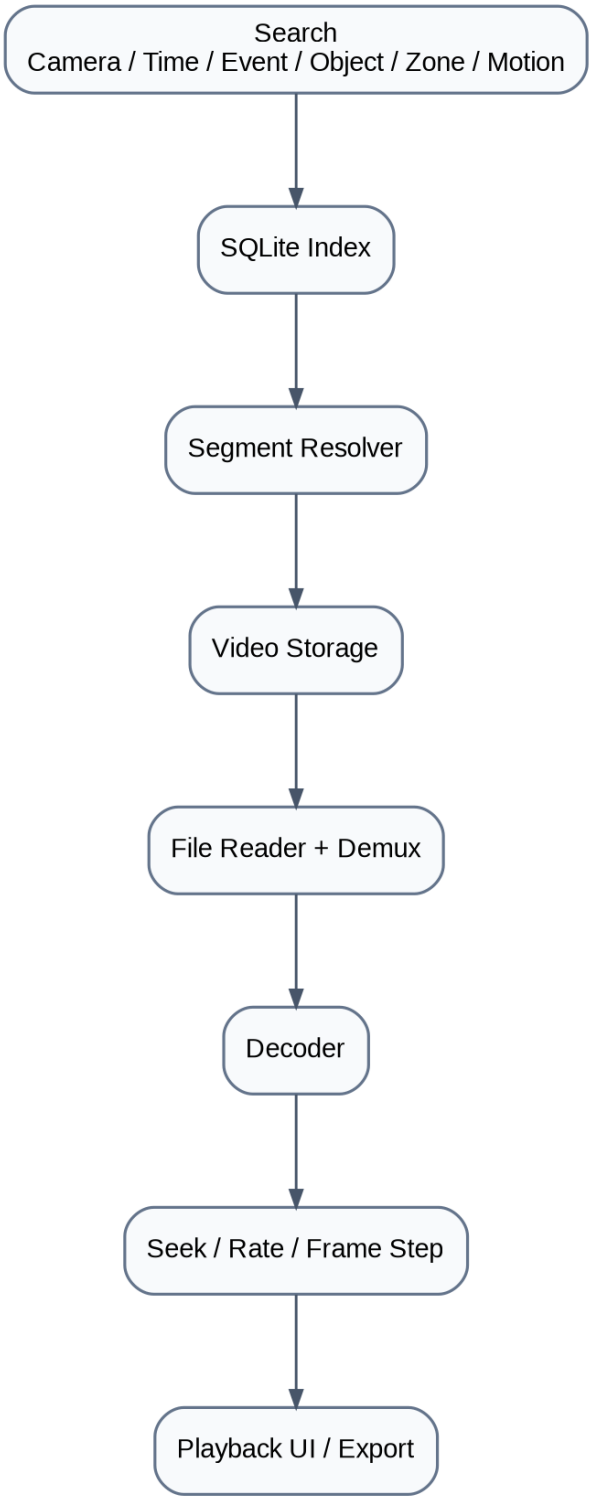
Pipeline diagram: 11. Event and Rule Pipeline

How it works

- Receive detection/tracking results or system state.
- Maintain temporal state.
- Apply the configured rule.
- Open/update/close the event.
- Store event metadata.
- Reference the relevant recording segment.
- Send notification if policy requires it.

12. Playback, Search and Export

Playback first finds the right recording using metadata and then reads the actual video segment. Recording and playback are separate workloads.



Pipeline diagram: 12. Playback, Search and Export

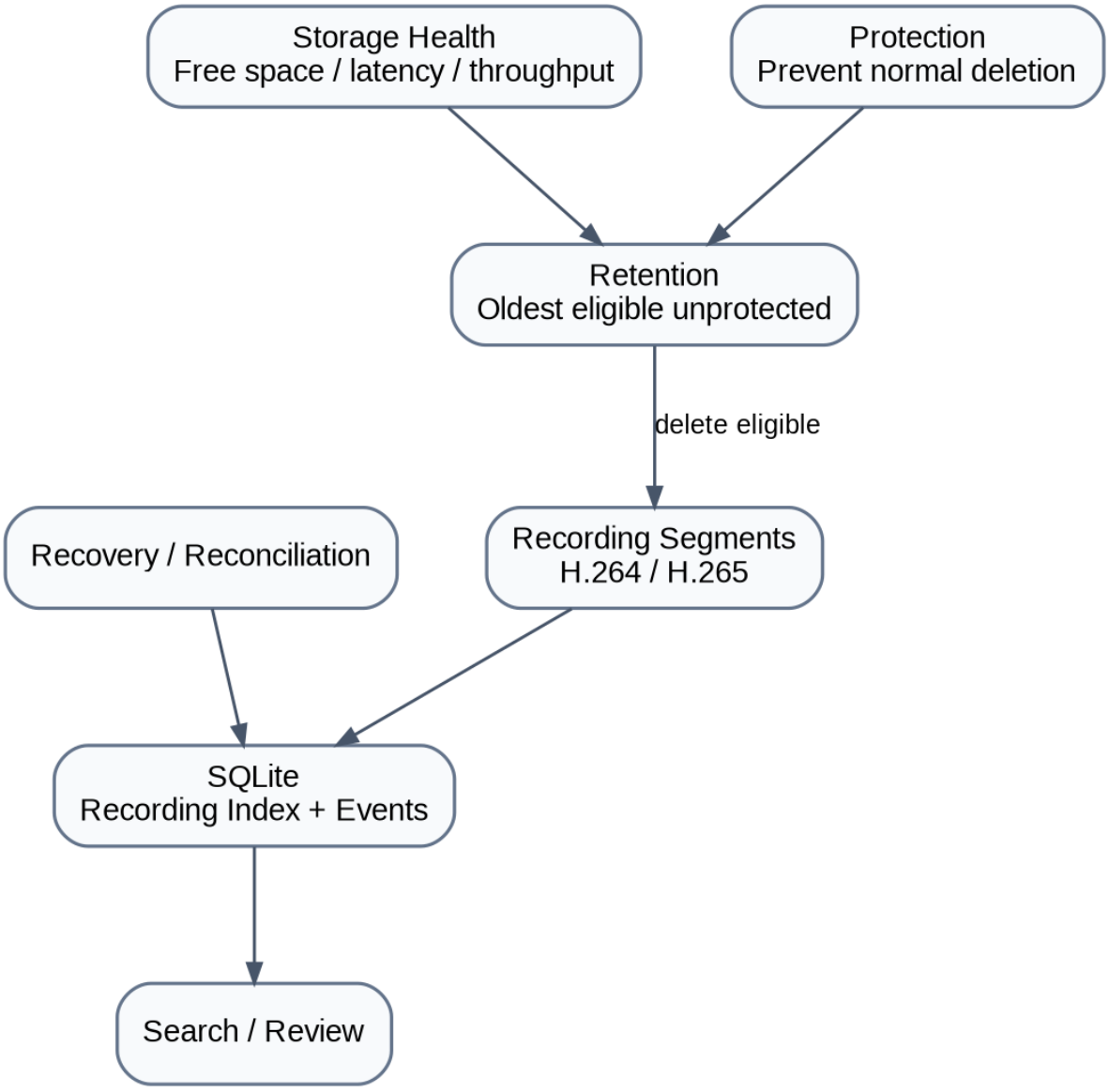
How it works

- User supplies camera/time/event filters.
- SQLite returns matching events or segment references.

- Playback opens the selected segment.
- Demux and decode.
- Seek to the requested timestamp.
- Render locally or prepare export.

13. Storage and Metadata

Video files hold the actual H.264/H.265 recordings. SQLite holds the information needed to find, protect and manage those recordings.



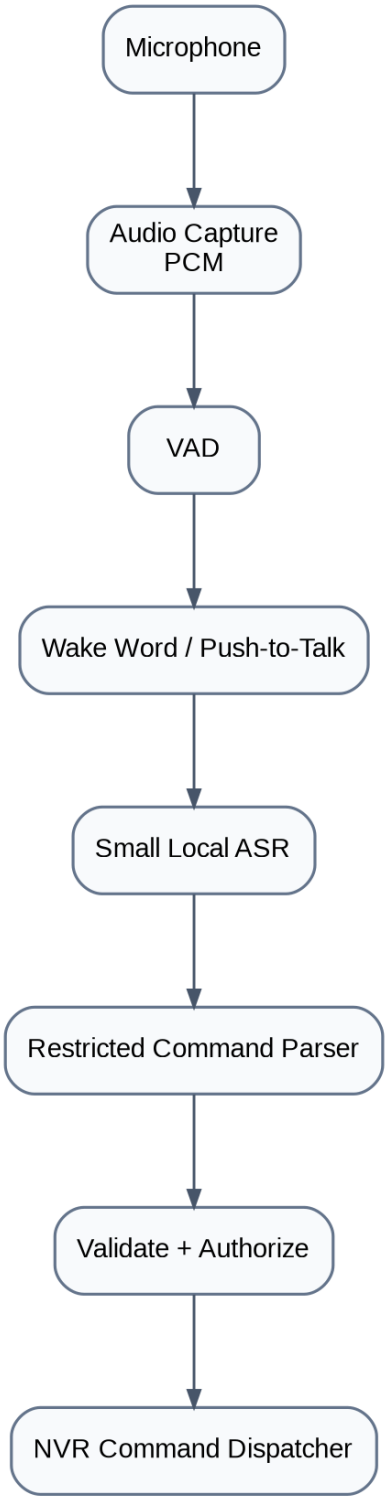
Pipeline diagram: 13. Storage and Metadata

How it works

- Recorder creates segments.
- Storage manager checks capacity and health.
- Segment path and time range are indexed.
- Events/detections/tracks reference recordings.
- Retention removes only eligible footage.
- Protected footage is not silently deleted.

14. Voice Control Pipeline

Voice is a separate process. It converts speech into a structured command and sends that command through the same validation and authorization path used by the UI and API.



Pipeline diagram: 14. Voice Control Pipeline

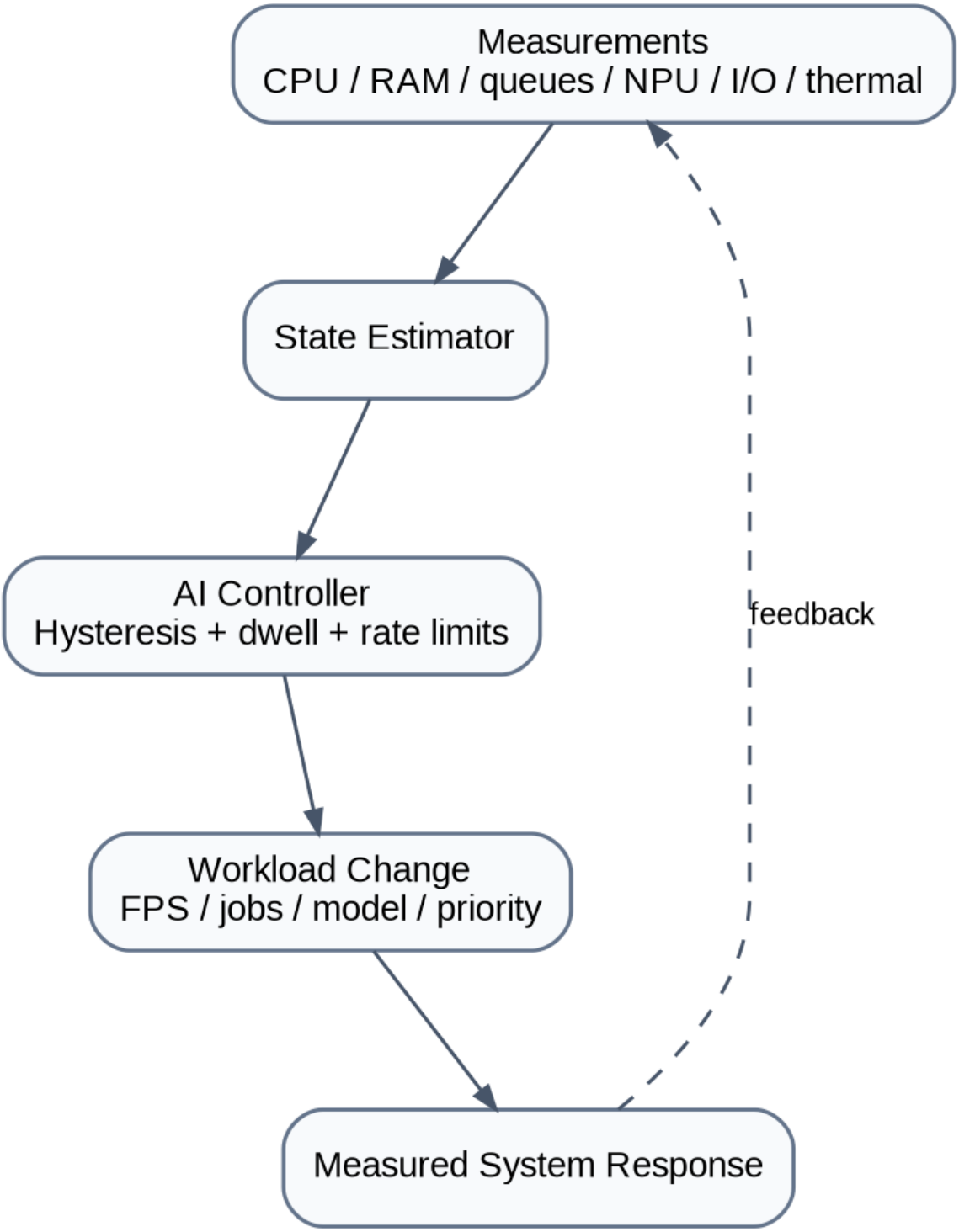
How it works

- Capture microphone audio.
- VAD/wake word or push-to-talk.

- ASR converts speech to text.
- Parser creates intent and parameters.
- Validate command.
- Check authorization.
- Send structured command to dispatcher.
- Execute normal NVR operation.

16. Resource and Health Management

Resource management is a feedback loop: measure the system, detect pressure, reduce lower-priority work, then measure again.



Pipeline diagram: 16. Resource and Health Management

How it works

- Measure CPU, RAM, decoder, AI, network, storage and thermal state.
- Estimate current workload state.

- Apply workload policy.
- Reduce AI/background work when required.
- Keep protected services within their limits.
- Record health metrics for diagnosis.

15. UI, API and Unified Command Flow

The UI, REST/WebSocket API and voice service all reach the same command dispatcher. This is important because the permission and validation rules are applied in one place.

Example: the UI sends ENABLE_AI(camera=4, model=person, requested_fps=5). The dispatcher validates the camera and model, checks the user's permission, then passes the request to the AI engine. Voice uses exactly the same execution path after speech has been converted to structured parameters.

17. Data Flow Between Pipelines

Data	Origin	Main consumers	Where it ends
Compressed H.264/H.265	Camera MAIN/SUB	Recording, stream consumers	Recording segments / live/AI acquisition
Decoded video frames	Hardware decoder	Live, AI, fullscreen, playback	Display, inference or playback
Detection metadata	AI inference	Tracker, event engine	SQLite detection/event records
Event state	Event/rule engine	UI, notifications, search	SQLite + notification targets
Recording metadata	Recorder	Search, playback, retention	SQLite segment index
Commands	UI/API/voice	Command dispatcher	Camera/recording/AI/playback/storage/etc.
Health measurements	Core components	Resource/health manager	Health history + control decisions

18. How Developers Should Implement It

Implement from the media path outward. Do not start by implementing the UI or AI rules first. The camera and recording path must be stable because almost every other feature depends on it.

Step 1 — Camera input

Implement RTSP/RTP acquisition, stream session ownership, codec/timestamp handling, reconnect and camera health.

Step 2 — Recording

Implement compressed MAIN recording, segmentation, safe finalization, SQLite segment indexing and startup reconciliation.

Step 3 — SUB decode

Add hardware decode for the SUB stream and prove stable 4/8-camera live operation.

Step 4 — Live UI

Connect the UI as a client. The UI requests views; it does not open camera streams itself.

Step 5 — Fullscreen

Add MAIN reuse/creation and fullscreen decode behavior.

Step 6 — AI

Add frame mailbox, motion gate, scheduler, preprocessing, inference, postprocessing and tracking.

Step 7 — Events

Convert AI/system results into temporal events and metadata.

Step 8 — Playback/search

Use SQLite metadata to locate segments, then implement demux/decode/seek/export.

Step 9 — Storage policy

Add retention, protection, low-storage states and recovery behavior.

Step 10 — Voice/API/security

Connect all control paths to the same dispatcher and RBAC checks.

Step 11 — Resource controller

Add measurements first; only then enable adaptive AI/background workload changes.

Step 12 — Qualification

Run the defined multi-camera, AI, playback, storage, failure and long-duration tests on the target hardware.

Core interfaces to keep hardware-independent

- IVideoDecoder — hardware/software decoder abstraction.
- IVideoScaler — resize and color conversion abstraction.
- IInferenceBackend — AI model execution abstraction.
- IBufferAllocator — frame/surface allocation abstraction.
- IDisplayBackend — display/compositor abstraction.
- IAudioBackend — microphone/audio abstraction.
- IStorageBackend — storage operations abstraction.
- IStreamGateway — optional remote stream gateway abstraction.

19. Important Developer Rules

- Recording must not depend on AI. If AI stops, recording should continue.
- UI must not own RTSP acquisition. Camera sessions belong to NVR core.
- Do not create unnecessary duplicate camera sessions. Use stream keys and session ownership/reference tracking.
- AI queues must be bounded. Prefer fresh frames over processing stale frames.
- Do not use SQLite for raw video frames. Store metadata and references only.
- Do not silently delete protected footage. Storage pressure must move through defined states.
- Keep time domains separate. Do not mix monotonic latency time with wall-clock event time.
- Voice never bypasses authorization. Voice commands use the same dispatcher and RBAC.
- Measure before optimizing. Hardware acceleration, zero-copy, decoder sharing and NPU concurrency must be verified on the target platform.
- Protect the media paths. Slow remote clients, AI overload or UI problems must not create unbounded pressure on recording.

20. Resource Validation and Medium-Detail Mathematics

The formulas below are intentionally kept at medium detail: enough for a developer to calculate and validate the system, but without turning the implementation guide into a mathematical specification.

20.1 CPU utilization

$$CPU_{total} = CPU_{RTSP} + CPU_{record} + CPU_{decode} + CPU_{copy} + CPU_{AI\text{-}pre} + CPU_{AI\text{-}post} + CPU_{tracking} + CPU_{storage} + CPU_{UI} + CPU_{IPC} + CPU_{logging} + CPU_{security} + CPU_{scheduler} + CPU_{other}$$

$$\text{CPU utilization (\%)} = \text{Total CPU time} / (\text{Number of logical CPUs} \times \text{measurement time}) \times 100$$

Acceptance target: sustained CPU utilization should remain below 50% under the defined full workload. Also record peak/critical-core utilization, iowait and scheduler pressure; total average alone is not sufficient.

20.2 Network bandwidth

$$B_{payload} = \Sigma B_i$$

$$B_{wire} = B_{payload} / \eta$$

B_i is the measured camera bitrate. η represents protocol/link efficiency after overhead. Measure camera ingress and remote egress separately.

20.3 Storage per day

$$D_i = (B_i \times 86{,}400) / 8$$

$$D_{day} = (86{,}400 / 8) \times \Sigma B_i$$

$$D_{retention} = T_{days} \times D_{day}$$

$$\text{For variable bitrate: } D_i = (1 / 8) \times \int_0^{86{,}400} B_i(t)\, dt$$

Example only: eight constant 4 Mb/s streams produce 345.6 GB/day before filesystem/container/metadata/reserved-capacity overhead. This is arithmetic, not a camera bitrate assumption.

20.4 Frame memory

$$M_{frame} = W \times H \times Bpp$$

$$M_{decoded} = N \times Q \times W \times H \times Bpp$$

$$\text{NV12 visible-sample example: } 1920 \times 1080 \times 1.5 = 3{,}110{,}400 \text{ bytes} \approx 3.11 \text{ MB}$$

For physical allocation, use actual stride/alignment and allocator metadata:

$$M_{surface} = stride_Y \times H + stride_{UV} \times (H / 2) + metadata + allocator\ overhead$$

20.5 Application memory budget

$$M_{application\ budget} = M_{usable} - M_{reserved}$$

$$\text{Acceptance: } M_{peak} < M_{available}$$

$$M(t) = M_0 + r \times t + \varepsilon(t)$$

$$\text{Leak criterion: } |r| < \varepsilon_M$$

Measure OS/kernel use, reserved/CMA/GPU/NPU memory where applicable, application heap, decoder surfaces, AI tensors, UI buffers, SQLite/page cache and network buffers. The memory test should show no unbounded growth under the full workload.

20.6 AI capacity

$$\rho = [\Sigma (\lambda_i \times s_i)] / K$$

λ_i is the inference arrival rate of job i in inferences/second. s_i is measured inference service time in seconds/inference. K is the number of equivalent parallel execution slots. For heterogeneous CPU/NPU/GPU resources, account for each resource separately.

Require ρ < 1 with engineering headroom

Also measure AI frame age and P95/P99 inference latency. A mathematically available slot is not useful if the frame is already stale.

20.7 Queue capacity

$$Q \le Q_{max}$$

Every important queue should have a configured maximum size, overflow behavior, age limit, backpressure rule and counters for drops/stale work. Recording queues are protected; AI/live queues may drop stale frames according to policy.

20.8 AI end-to-end latency

$$L_{AI} = L_{capture} + L_{network} + L_{jitter} + L_{decode} + L_{sampling} + L_{motion} + L_{preprocess} + L_{inference} + L_{postprocess} + L_{tracking} + L_{event}$$

Report P50, P95 and P99. The tail values matter because occasional long delays can make an otherwise good average misleading.

20.9 Storage throughput

Sustained recording write rate < sustained storage write capability

Validate the actual write path with the intended filesystem, segment size, concurrent metadata activity and storage device. Do not rely only on a peak benchmark number.

21. Math Terms — Short Definitions

- B_i — bitrate of camera i, measured in bits/second.
- $B_{payload}$ — sum of camera media bitrates before protocol overhead.
- η — efficiency factor representing protocol/link overhead.
- D_i — storage used by one camera over one day.
- T_{days} — required retention period in days.
- W, H — frame width and height in pixels.
- Bpp — bytes used per pixel for the selected frame representation.
- N — number of cameras.
- Q — buffered/decoded surfaces or queue occupancy, depending on the formula.
- M_{peak} — highest measured application memory use.
- r — measured steady-state memory growth rate.
- λ — AI inference arrival rate in jobs/second.
- s — measured AI service time per inference.
- K — number of equivalent parallel AI execution slots.
- ρ — AI offered load compared with available execution capacity.

- P50 — median; half of measurements are at or below it.
- P95 — 95% of measurements are at or below it.
- P99 — 99% of measurements are at or below it.
- Q_{max} — configured maximum queue size.
- L — latency from one pipeline stage to the corresponding output.

22. Validation Checklist

The architecture is selected here; final performance claims require runtime evidence on the target hardware.

Test	What to verify
1 camera	Connect, record, play back, decode, AI and reconnect.
4 cameras	Stable recording and live grid; measure CPU/RAM/network/storage.
8 cameras recording	Continuous recording, segment integrity and storage throughput.
8 cameras + live	Live freshness without recording degradation.
8 cameras + 2 AI	AI admission, latency and resource use.
8 cameras + 3 AI	AI capacity and resource headroom.
8 cameras + playback	Playback must not block protected recording.
8 cameras + voice	Voice must not disturb media paths.
AI overload	Stale AI frames are dropped/replaced and workload is reduced.
Storage stall	Recovery works within the defined window; prolonged failure constrains recording without silently
Camera reboot / codec reconnect	Camera reconnects and produces a valid new recording segment.
Slow remote client	Remote backpressure does not block local protected paths.
24 h / 72 h	No unbounded memory growth, persistent queue growth, or unexplained resource drift.

23. References and Resources

These references are retained from the earlier research and architecture work. They are included so developers can inspect the original documentation and papers behind the selected pipeline patterns.

[N1] Frigate — Video Pipeline — Main/substream model, motion processing, decoding, detection and review concepts.
https://docs.frigate.video/frigate/video_pipeline/

[N2] Frigate — Hardware Acceleration — Hardware-decoding configuration and platform verification.
https://docs.frigate.video/configuration/hardware_acceleration_video/

[N3] Frigate — Recording — Recording and export behavior.
https://docs.frigate.video/configuration/record/

[N4] Frigate — Restream — Stream reuse/restream reference.
https://docs.frigate.video/configuration/restream/

[N5] Frigate — Camera Configuration — Separate recording/detect stream roles.
https://docs.frigate.video/configuration/cameras/

[N6] Frigate — Face Recognition — Staged local face detection/embedding concepts.
https://docs.frigate.video/configuration/face_recognition/

[N7] Frigate — License Plate Recognition — Vehicle-first LPR flow and OCR integration.
https://docs.frigate.video/configuration/license_plate_recognition/

[Z1] ZoneMinder — Components — NVR component structure and streaming/processing roles.
https://zoneminder.readthedocs.io/en/latest/userguide/components.html

[Z2] ZoneMinder — Recording — Original-stream/passthrough recording concepts.
https://zoneminder.readthedocs.io/en/latest/userguide/definemonitor/definemonitor_recording.html

[Z3] ZoneMinder — Stream Socket — Streaming/IPC reference.
https://zoneminder.readthedocs.io/en/latest/stream_socket.html

[S1] Shinobi — Detection — Detection, events and event filtering.
https://docs.shinobi.video/detect

[S2] Shinobi — Detector Plugin — Detector abstraction.
https://docs.shinobi.video/create/detector-plugin

[S3] Shinobi — Developer — Developer architecture/API reference.
https://docs.shinobi.video/developer

[G1] go2rtc — Stream broker/restream and protocol support.
https://github.com/AlexxIT/go2rtc

[G2] GStreamer Application Development — Application pipeline construction and control.
https://gstreamer.freedesktop.org/documentation/application-development/

[G3] GStreamer rtspsrc — RTSP source, RTP session, jitter, reordering and timing.
https://gstreamer.freedesktop.org/documentation/rtsp/rtspsrc.html

[G4] GStreamer tee — Branching a media stream to multiple consumers.
https://gstreamer.freedesktop.org/documentation/coreelements/tee/

[G5] GStreamer queue — Queue limits, threading and monitoring.
https://gstreamer.freedesktop.org/documentation/coreelements/queue/

[G6] GStreamer splitmuxsink — Time/size-based segmenting and keyframe-aware behavior.
https://gstreamer.freedesktop.org/documentation/multifile/splitmuxsink.html

[G7] GStreamer DMA-BUF — Linux video-buffer sharing between compatible components.
https://gstreamer.freedesktop.org/documentation/additional/design/dmabuf.html

[G8] GStreamer Buffer Pools — Buffer pool and allocation behavior.
https://gstreamer.freedesktop.org/documentation/additional/design/bufferpool.html

[G9] GStreamer Shared Memory — Shared-memory transport reference.
https://gstreamer.freedesktop.org/documentation/shm/

[O1] ONVIF Profiles — Profiles overview for IP-camera functions.
https://www.onvif.org/profiles/

[O2] ONVIF Profile T — Advanced IP-video streaming, H.264/H.265, events, metadata and PTZ.
https://www.onvif.org/profiles/profile-t/

[O3] ONVIF Profile G — Edge storage and retrieval.
https://www.onvif.org/video/ovif-profile-g-for-edge-storage-and-retrieval/

[O4] ONVIF Profile M — Analytics metadata and events.
https://www.onvif.org/profiles/profile-m/

[O5] ONVIF Profile S — Basic IP-video streaming/configuration reference.
https://www.onvif.org/profiles/profile-s/

[A1] NoScope — Cheap temporal filtering before expensive neural processing.
https://arxiv.org/abs/1703.02529

[A2] Focus — Efficient indexing/specialized processing to reduce video query cost.
https://arxiv.org/abs/1801.03493

[A3] OC-SORT — Lightweight observation-centric multi-object tracking candidate.
https://arxiv.org/abs/2203.14360

[A4] Adaptive edge video analytics research — Dynamic resource allocation and adaptive processing.
https://xplorestaging.ieee.org/document/11640880

[A5] Online resolution adaptation/resource allocation — Adaptive adjustment of video-analysis workload.
https://www.sciencedirect.com/science/article/pii/S1389128624001749

[A6] Surveillance storage/keyframe indexing research — Efficient video retrieval/indexing concepts.
https://onlinelibrary.wiley.com/doi/10.1155/2018/3839104

[SQA1] SQLite Write-Ahead Logging — WAL behavior and checkpointing.
https://sqlite.org/wal.html

[SQA2] SQLite FTS5 — Optional full-text search extension.
https://www.sqlite.org/fts5.html

[AI1] CompreFace — Open-source face-recognition service/API reference.
https://github.com/exadel-inc/CompreFace

[AI2] PaddleOCR — Open-source OCR stack.
https://github.com/PaddlePaddle/PaddleOCR

[AI3] Tesseract OCR — Open-source OCR engine.
https://github.com/tesseract-ocr/tesseract

24. What Comes From Each Reference

Reference family	What is used as a practical reference in this guide
Frigate	Main/detection stream separation, motion-before-expensive-AI pattern, hardware decode awareness, recor
ZoneMinder	NVR component separation, streaming/processing roles and compressed/passthrough recording concepts.
Shinobi	Detector abstraction and event filtering concepts.
go2rtc	Stream brokerage/restream concepts; treated as an optional implementation rather than the NVR business
GStreamer	RTSP/RTP acquisition, branching, bounded queues, segmentation, buffer management and hardware-comp
ONVIF	Camera discovery, profiles, PTZ and analytics/event interfaces where supported by the camera.
Research papers	Temporal filtering, video query efficiency, lightweight tracking and adaptive edge-video resource manager
SQLite	WAL/checkpointing and optional FTS5 for metadata/search workloads.

Scope note: These references inform the implementation patterns. They do not replace target-board qualification. Hardware-specific throughput, decoder concurrency, zero-copy behavior, NPU concurrency, thermal limits, filesystem performance and exact queue sizes must be measured and recorded during implementation.